

AI Pediatric Pneumonia Detector

Technical Documentation Report

Module:	Chatbot System & PDF Report Generator (agentt/)
Version:	v9
Institution:	University of Saida, Algeria
Generated:	April 25, 2026
Report Type:	Architecture & Implementation Reference

Table of Contents

1.	Executive Summary	3
2.	System Overview & Purpose	3
3.	System Architecture	4
4.	Module: layer1.py — Document Processor	5
5.	Module: retrieval.py — Hybrid Retrieval Engine	6
6.	Module: prompt_builder.py — Prompt Construction	8
7.	Module: layer2.py — Chatbot Orchestration	9
8.	Module: report_agent.py — PDF Report Generator	10
9.	Module: med_kb.py — Medical Knowledge Base	12
10.	Module: agent_utils.py — Shared Utilities	13
1.		
1.	End-to-End Execution Flow	14
2.	Data Flow Diagram (Textual)	15
3.	Dependencies & Environment Requirements	16
4.	Configuration & API Keys	17
5.	Medical Knowledge Base (references/ folder)	18
6.	Security & Data Handling	18
7.	Limitations & Future Work	19

1. Executive Summary

This document provides a comprehensive technical reference for the Chatbot System and PDF Report Generator modules implemented within the **AI Pediatric Pneumonia Detector** application (version 9). These two components — collectively housed in the **agentt/** directory — extend the application's core diagnostic capabilities by adding intelligent conversational reasoning and automated medical report generation.

The chatbot operates as a clinical reasoning engine: it ingests patient-specific documents (PDFs) uploaded by the treating physician, retrieves semantically relevant context from those documents and from a curated medical knowledge base, constructs a richly structured prompt, and sends this prompt to a Large Language Model (LLM) via the OpenRouter API. The LLM returns a dense, clinically actionable response structured around risk identification, reasoning, and recommended action.

The report generator reads all patient data stored in the application's file-based data store — diagnostics, treatment plans, hospitalisation records, nurse monitoring tables, and uploaded documents — and compiles a professional A4 PDF report using the ReportLab library. Both components are designed to integrate seamlessly with the existing Streamlit-based user interface.

2. System Overview & Purpose

The AI Pediatric Pneumonia Detector is a clinical decision-support application designed for pediatric physicians. Its primary function is to assist in the diagnosis and management of pneumonia in children through AI-powered analysis of vital signs, clinical symptoms, and chest X-ray data. Version 9 introduces two new intelligent agent modules:

2.1 Chatbot System (Conversational Clinical Reasoner)

- Accepts physician-authored queries about a specific patient.
- Processes uploaded patient PDF documents through a multi-stage retrieval pipeline.
- Retrieves supporting evidence from a pre-built Medical Knowledge Base (MedKB).
- Constructs structured prompts following strict clinical reasoning rules.
- Queries an LLM (via OpenRouter API, compatible with GPT-4, Claude, Mistral, etc.).
- Returns concise, evidence-grounded clinical insights formatted for immediate clinical use.

2.2 PDF Report Generator (Automated Medical Documentation)

- Reads all structured patient data from the doctor's folder (CSV files, check tables, etc.).
- Aggregates diagnostic sessions, treatment plans, hospitalisation records, and uploaded files.
- Produces a professionally formatted, paginated A4 PDF report using ReportLab.
- Embeds physician and patient identification, colour-coded diagnostic tables, discharge criteria, and a full audit trail.
- Saves the report directly into the patient's folder for record-keeping.

Together, these modules transform the application from a diagnostic tool into a complete clinical workflow system, supporting both real-time decision support and formal medical documentation.

3. System Architecture

The agentt/ module follows a layered architecture with clear separation of concerns. The system can be conceptually divided into five layers:

Layer	Components	Responsibility
Layer 1 (Ingestion)	layer1.py	Parse uploaded PDFs, chunk text, build vector + BM25 indexes
Layer 2 (Retrieval)	retrieval.py	Hybrid vector + BM25 search over patient KB and medical KB; reranking
Layer 3 (Reasoning)	prompt_builder.py	Assemble structured prompt from retrieved chunks + doctor query
Layer 4 (Generation)	layer2.py (LLM call)	Send prompt to OpenRouter LLM, receive and stream response
Layer 5 (Reporting)	report_agent.py	Read all patient data, generate structured PDF report
Support	med_kb.py, agent_utils.py	Medical knowledge base construction; shared data-access utilities

3.1 Key Architectural Principles

- **Dual Knowledge Sources:** Every query is answered using BOTH patient-specific content (uploaded PDFs) and generalised medical literature (MedKB), ensuring responses are both personalised and evidence-based.
- **Hybrid Retrieval:** Combining dense vector search (semantic similarity via FAISS) with sparse keyword search (BM25) captures both conceptual and exact-term matches, improving recall significantly over single-method approaches.
- **Reranking Layer:** A cross-encoder reranker (FlashRank or Cohere) re-scores retrieved candidates for precision, reducing noise before passing context to the LLM.
- **Prompt Engineering:** The prompt_builder enforces strict clinical output formatting, preventing the LLM from returning vague or non-actionable responses.
- **File-Based Data Store:** Patient data is stored in structured CSV files and subdirectories rather than a database, enabling easy portability and human-readable records.
- **Cache-Aware MedKB:** The medical knowledge base is built once from reference PDFs/TXTs, cached as a pickle file, and reloaded on subsequent runs — avoiding costly re-computation.

3.2 Integration with the Main Application

The agentt/ modules are integrated into the main Streamlit application through direct Python imports. The Streamlit UI provides the entry point for physician interaction: uploading patient documents, submitting clinical queries, triggering report generation, and displaying LLM responses. The agent_utils.py module reads the active doctor session from a shared session file (apppy/active_doctor), enabling the agents to locate the correct patient data folder without requiring explicit path configuration at call time.

4. Module: layer1.py — Document Processor

File: agentt/layer1.py **Role:** Ingestion Layer — converts raw PDF/text files into searchable indexes.

This module defines the **DocumentProcessor** class, which is the entry point for processing any document uploaded by the physician during a chatbot session. It handles three distinct tasks: text extraction, chunking, and dual-index construction.

4.1 Class: DocumentProcessor

Constructor (`__init__`)

Initialises three components on instantiation:

- **SentenceTransformer('all-MiniLM-L6-v2')**: A lightweight, high-performance sentence embedding model (384-dimensional vectors). Used to encode text chunks into dense vector representations for semantic search.
- **RecursiveCharacterTextSplitter**: LangChain's text splitter configured with `chunk_size=800` characters and `chunk_overlap=100` characters. The overlap prevents context loss at chunk boundaries.
- **Ranker() (FlashRank)**: A local cross-encoder reranker that scores candidate passages against a query for precision retrieval. Requires no API key.

Method: `process_file(file) -> dict`

Accepts a file-like object (from Streamlit's `file_uploader`). Supports PDF and plain-text formats.

- For PDFs: uses `pypdf.PdfReader` to extract text page-by-page, concatenating with newline separators.
- For text files: decodes bytes to UTF-8 string.
- Splits the extracted text into overlapping chunks using the `RecursiveCharacterTextSplitter`.
- Encodes all chunks using the `SentenceTransformer` to produce a NumPy embedding matrix.
- Builds a **FAISS IndexFlatL2** index from the embedding matrix for exact L2 nearest-neighbour search.
- Tokenises chunks (lowercase split) and builds a **BM25Okapi** index for keyword-based retrieval.
- Returns a dictionary: `{'chunks': list[str], 'index': faiss.Index, 'bm25': BM25Okapi}`.

Method: `hybrid_search_and_rerank(query, kb, top_k=3) -> list[str]`

Implements the full retrieval pipeline for a single knowledge base:

- Encodes the query into a vector and runs FAISS search to retrieve `top_k*2` candidates by semantic similarity.
- Runs BM25 keyword search to retrieve `top_k*2` candidates by term overlap.
- Combines and deduplicates both candidate lists using Python set operations.
- Constructs `RerankRequest` passages and calls FlashRank's reranker to produce a relevance-ordered list.
- Returns the `top_k` highest-scoring chunks as plain strings.

4.2 Dependencies

Import	Package	Purpose
<code>PdfReader</code>	<code>pypdf</code>	PDF text extraction
<code>RecursiveCharacterTextSplitter</code>	<code>langchain_text_splitters</code>	Intelligent text chunking with overlap
<code>SentenceTransformer</code>	<code>sentence-transformers</code>	Dense text embedding

faiss	faiss-cpu	Vector similarity index
numpy	numpy	Array operations for FAISS
BM25Okapi	rank-bm25	Sparse keyword search
Ranker, RerankRequest	flashrank	Local cross-encoder reranking

5. Module: retrieval.py — Hybrid Retrieval Engine

File: agentt/retrieval.py **Role:** Retrieval Layer — unified hybrid search across both knowledge bases.

This module implements the advanced retrieval pipeline that queries both the patient-specific knowledge base (built from uploaded PDFs) and the medical knowledge base (built from reference papers) simultaneously. Unlike layer1.py which handles a single KB, retrieval.py orchestrates cross-KB retrieval with configurable reranking strategies.

5.1 Module-Level Configuration

On import, the module detects available optional dependencies:

- **COHERE_AVAILABLE:** Set to True only if the cohere package is installed AND the COHERE_API_KEY environment variable is set. Enables cloud-based reranking via Cohere's rerank-english-v3.0 model.
- **FLASHRANK_AVAILABLE:** Set to True if the flashrank package is importable. Enables local reranking without any API dependency.

This conditional detection implements a **graceful degradation pattern**: Cohere (best quality) → FlashRank (good, local) → Simple keyword overlap (fallback).

5.2 Internal Search Functions

_vector_search(query_vec, kb, top_k) -> list[dict]

Performs FAISS vector search on a knowledge base's FAISS index. Accepts a pre-encoded query vector (NumPy array). Returns up to top_k chunk dictionaries from kb['chunks']. Handles edge cases: empty index or empty chunk list returns [].

_bm25_search(query, kb, top_k) -> list[dict]

Performs BM25 keyword search on the user knowledge base only. Tokenises the query by lowercase splitting, computes BM25 scores across all chunks, and returns top_k chunks with positive scores. The medical KB intentionally excludes BM25 (marked 'too large' in code comments) to avoid false keyword matches.

_rerank_cohere(query, candidates, top_k) -> list[dict]

Calls the Cohere Rerank API (rerank-english-v3.0 model) with all candidate chunks. Returns top_k results enriched with a 'score' field containing the relevance score from Cohere. Requires COHERE_API_KEY environment variable.

_rerank_flashrank(query, candidates, top_k) -> list[dict]

Instantiates a FlashRank Ranker and submits a RerankRequest. Passages are indexed by position and mapped back to original chunk dicts after reranking. Returns top_k chunks with scores.

_rerank_simple(query, candidates, top_k) -> list[dict]

Fallback reranker using Jaccard-like word overlap between the query and each candidate's text. Scores are computed as: $\text{len}(\text{query_words} \cap \text{text_words}) / (\text{len}(\text{query_words}) + 1)$. Sorted descending; returns top_k. No external dependencies.

5.3 Main Function: hybrid_retrieve()

Signature: **hybrid_retrieve(query, user_kb, med_kb, embed_model, top_k_final=5, top_k_candidates=15)**
-> dict

This is the primary public interface of the module. Pipeline steps:

Step 1 — Query Encoding: The query string is encoded into a dense vector using the shared SentenceTransformer embed_model passed as a parameter. This ensures the same embedding space is used for both ingestion and retrieval.

Step 2 — Vector Search (Both KBs): FAISS search is run independently on user_kb (patient documents) and med_kb (medical literature), each retrieving top_k_candidates//2 (7 by default) candidates.

Step 3 — BM25 Search (User KB Only): BM25 search retrieves top_k_candidates//4 (3-4) keyword-matched chunks from the user KB only. This supplements semantic search with exact term matching.

Step 4 — Merge & Deduplicate: User KB candidates from vector and BM25 are merged. Deduplication is by exact text match using a Python set. Medical KB candidates are kept separate.

Step 5 — Combined Reranking: All user and medical candidates are concatenated into a single pool and passed to the best available reranker (Cohere > FlashRank > Simple). The reranker scores every candidate against the query in one pass.

Step 6 — Source-Tagged Splitting: Reranked results are split by their 'tag' field: chunks with tag='PDF' (patient documents) vs tag='MED' (medical literature). The top 3 from each source are returned.

Return Value:

Returns a dictionary: `{'pdf_chunks': list[dict], 'med_chunks': list[dict]}`. Each chunk dict contains at minimum: 'text' (content), 'source' (filename), 'tag' ('PDF' or 'MED'). Maximum 3 chunks per source in the default configuration.

6. Module: `prompt_builder.py` — Prompt Construction

File: `agentt/prompt_builder.py` **Role:** Reasoning Layer — assembles the structured LLM prompt.

This module contains a single function that constructs the complete prompt submitted to the LLM. It is the most clinically critical component of the chatbot pipeline, as it controls the LLM's behaviour, output format, and reasoning style.

6.1 Function: `build_final_prompt()`

Signature: `build_final_prompt(doctor_prompt, pdf_chunks, med_chunks) -> str`

=== ROLE ===

Establishes the LLM's persona as a 'clinical reasoning engine' — a senior pediatric physician. Critically, it instructs the model NOT to summarise records but to THINK beyond the raw data, combining patient facts with clinical knowledge to draw conclusions unavailable from the chart alone.

=== STRICT OUTPUT RULES ===

Enforces formatting constraints: responses must be under 200 words, must never repeat raw PDF data, must cite MED sources for every recommendation, and must follow a three-part structure: - RISK: one line identifying the most urgent finding - REASONING: 2-3 lines of clinical interpretation using MED knowledge - ACTION: 1-2 lines of specific, evidence-grounded recommendation

=== CLINICAL INFERENCE RULES ===

Provides 10 explicit inference patterns (e.g., 'antibiotic given: yes across 4 tables + condition worsening = treatment failure'). These rules encode clinical heuristics that guide the LLM to identify dangerous patterns that may not be immediately obvious from individual data points. Examples include: progressive cyanosis = emergency criterion; vomiting 60% of time = IV route mandatory; penicillin allergy cross-check; worsening trend across nurse tables = escalation needed.

=== DOCTOR INSTRUCTION ===

Injected only if the physician provided a query. Contains the physician's natural-language question or instruction verbatim. This section is the primary steering input for the LLM response.

=== PATIENT FACTS [PDF] ===

Injects the retrieved PDF chunk texts, numbered [PDF-1], [PDF-2], [PDF-3]. The section header explicitly labels these as 'USE FOR INFERENCE ONLY, DO NOT REPEAT' — preventing the LLM from paraphrasing chart data rather than analysing it.

=== MEDICAL KNOWLEDGE [MED] ===

Injects the retrieved MedKB chunk texts, numbered [MED-1], [MED-2], [MED-3], each prefixed with the source paper name. The section header labels these as mandatory justification sources. If no MED chunks are available, the LLM is instructed to notify the physician to add papers.

6.2 Design Rationale

The prompt design follows principles of chain-of-thought prompting combined with structured output constraints. By explicitly forbidding data repetition and mandating evidence citation, the system eliminates two common failure modes of medical LLM applications: (1) responses that merely restate what the physician already knows, and (2) recommendations without traceable justification. The clinical inference rules serve as few-shot examples of the reasoning depth expected.

7. Module: layer2.py — Chatbot Orchestration

File: agentt/layer2.py **Role:** Generation Layer — LLM API interaction and response streaming.

This module serves as the orchestration hub of the chatbot. It imports and coordinates all other components: document processing (layer1.py), hybrid retrieval (retrieval.py), and prompt construction (prompt_builder.py). It manages the Streamlit session state, handles file uploads, and communicates with the OpenRouter LLM API.

7.1 Session State Management

The module initialises and maintains Streamlit session state variables:

- **st.session_state.user_kb:** Dictionary storing the processed knowledge base for the currently uploaded patient PDF (chunks, FAISS index, BM25 index).
- **st.session_state.chat_history:** List of message dictionaries ({'role': 'user'|'assistant', 'content': str}) representing the conversation history.
- **st.session_state.processor:** The DocumentProcessor instance, initialised once per session to avoid reloading the embedding model on every interaction.

7.2 Document Upload Handling

A Streamlit file_uploader widget accepts PDF or text files from the physician. On upload, the file is passed to DocumentProcessor.process_file(), and the resulting knowledge base dict is stored in session state. A success confirmation is displayed to the physician.

7.3 Chatbot Query Processing Pipeline

1. Physician types a query in the Streamlit chat input widget.
2. The query is appended to chat_history as a user message.
3. hybrid_retrieve() is called with the query, user_kb (from session state), med_kb (the pre-built MedKB), and the shared embed_model.
4. The returned pdf_chunks and med_chunks are passed to build_final_prompt() along with the physician's query.
5. The complete prompt string is sent to the OpenRouter API using the openai Python client (OpenAI-compatible endpoint).
6. The API response is streamed token-by-token and displayed in real time in the Streamlit interface.
7. The complete response is appended to chat_history as an assistant message for conversation continuity.

7.4 LLM API Configuration

The module uses the **openai** Python library configured to point at the OpenRouter API base URL (<https://openrouter.ai/api/v1>). The API key is loaded from the OPENROUTER_API_KEY environment variable. OpenRouter is a unified API gateway that provides access to multiple LLM providers (OpenAI GPT-4, Anthropic Claude, Mistral, Meta Llama, etc.) through a single endpoint. The specific model is selected by the physician or configured in the application settings.

7.5 Medical Knowledge Base Integration

On startup, layer2.py instantiates the MedicalKnowledgeBase class from med_kb.py and calls its build() method. If a valid cache exists (med_kb_cache.pkl), the MedKB loads instantly. If not, it processes all PDFs and TXT files in the agentt/refs/ directory, builds FAISS and chunk structures, and saves the cache. The

MedKB object is stored globally for reuse across all chatbot sessions.

8. Module: report_agent.py — PDF Report Generator

File: agentt/report_agent.py **Role:** Reporting Layer — comprehensive patient PDF report generation.

This module provides a self-contained report generation system using the ReportLab library. It reads all patient data from the file-based store and compiles it into a multi-page, colour-coded A4 PDF document suitable for medical record-keeping and clinical handover.

8.1 Public Entry Point: generate_patient_report()

Signature: **generate_patient_report(doctor_folder, patient_folder_name) -> tuple[bool, str, Path|None]**

Returns a three-element tuple: (success_bool, message_str, pdf_path_or_None).

8.2 Data Loading Pipeline

The function calls agent_utils.py helpers to load all patient data:

Data Source	Function Called	Data Type
Patient registration	load_patient_csv()	Single-row dict (name, DOB, blood type, etc.)
Doctor profile	load_doctor_profile()	Single-row dict (name, degree, hospital)
Vital sign diagnostics	load_all_diagnostics()	Sorted list of diagnostic session dicts
Home treatment plans	load_home_treatments()	List of treatment plan dicts with medications
Hospitalisation record	load_hospitalized_treatment()	Single dict with diagnosis, notes, discharge criteria
Nurse monitoring tables	load_all_check_tables()	List of check table dicts with row-level data
X-ray files	list_xrays()	Sorted list of image Paths in xray/ subfolder
Uploaded PDFs	list_uploaded_pdfs()	Dict of subdirectory -> list of PDF Paths

8.3 Report Structure

Cover Page (_cover_block)

Full-width navy/blue header with application name. Patient and physician identification table in a 4-column grid. Colour: LIGHT (#EAF2FB) background with grey grid lines.

Section 1: Patient Registration (_patient_registration_section)

Two-column table listing email, phone, medical history, inherited illnesses, critical clinical notes, and registration date. Alternating white/grey row backgrounds.

Section 2: Vital Signs Diagnostic Timeline (_diagnostics_section)

Multi-column table with 11 columns: index, timestamp, temperature, SpO2, fever, cough, shortness of breath, confusion, prediction label, confidence, and truncated clinical notes. Prediction column is colour-coded: RED for 'Sick', GREEN for 'Not Sick'.

Section 3: Home Treatment Plans (_treatment_section)

For each plan: medication table (name, dosage, schedule, duration) with ORANGE header, followed by appointment date, follow-up notes, and a warning signs table with RED header.

Section 4: Hospitalisation Record (`_hospital_section`)

Two-column key-value table for diagnosis, admission notes, instructions, progress notes, discharge condition. A discharge readiness checklist with GREEN (MET) / ORANGE (PENDING) status cells. Summary of nurse monitoring check tables.

Section 5: X-Ray Studies (`_xray_section`)

Count of stored X-ray image files with filenames listed.

Section 6: Uploaded Medical Documents (`_uploaded_docs_section`)

Lists files across three subdirectories: `medical_history_pdf/`, `family_history_pdf/`, `clinical_notes_pdf/`.

Footer

Horizontal rule followed by a generation summary line. Running page footer via canvas callback: application name, institution, page number.

8.4 Colour Scheme

Constant	Hex Value	Usage
NAVY	#1B2A4A	Primary header backgrounds, section headings, discharge table header
BLUE	#2E6DA4	Sub-headers, horizontal rules, diagnostic table header
LIGHT	#EAF2FB	Cover block info table background, highlight boxes
GREEN	#27AE60	'Not Sick' prediction cells, met discharge criteria
RED	#C0392B	'Sick' prediction cells, warning signs table header
ORANGE	#E67E22	Medication table header, pending discharge criteria
GREY	#F5F5F5	Alternating table row backgrounds

8.5 Output

The PDF is saved to: `<patient_folder>/agent_report_YYYYMMDD_HHMMSS.pdf`. The timestamp suffix ensures no existing reports are overwritten, creating a chronological audit trail of generated reports. The Streamlit UI receives the path and offers a download button.

9. Module: med_kb.py — Medical Knowledge Base

File: agentt/med_kb.py **Role:** Support Layer — build, cache, and serve the medical literature knowledge base.

9.1 Class: MedicalKnowledgeBase

Encapsulates the complete lifecycle of the medical knowledge base: reading source files, cleaning and chunking text, building a FAISS index, caching, and serving search results.

9.2 Key Methods

build()

Scans the agentt/refs/ directory for all .pdf and .txt files. Computes an MD5 hash of filenames and modification times to detect changes. If a valid cache exists at agentt/med_kb_cache.pkl with a matching hash, loads chunks and FAISS index from cache without reprocessing. Otherwise, processes all files, builds the index, and saves a new cache.

search(query, top_k=3) -> list[dict]

Encodes the query using the embedded SentenceTransformer and runs FAISS L2 search. Returns up to top_k chunk dicts from the medical literature. Used directly by retrieval.py as an alternative to the full hybrid_retrieve() pipeline for simple MedKB-only queries.

_hash_dir(files) -> str

Computes an MD5 hash over file names and modification timestamps. This hash is stored in the cache and compared on reload to determine whether reprocessing is needed.

_read_file(path) -> str

Reads PDF files using pypdf (if available) or plain text files with UTF-8 encoding. Returns the extracted text string.

_clean(text) -> str

Applies several cleaning transformations: removes the References section, funding acknowledgements, conflict-of-interest statements, and DOIs. Expands medical abbreviations using a 17-entry ABBREV_MAP (e.g., 'CKD' -> 'Chronic Kidney Disease', 'SpO2' -> 'Oxygen Saturation'). Collapses excessive whitespace.

_chunk(text, source, max_chars=1600) -> list[dict]

Splits text on paragraph boundaries (double newline), accumulating paragraphs into chunks of up to 1600 characters. Each chunk dict contains: 'text', 'source' (filename stem), and 'tag': 'MED' — the tag used by retrieval.py to distinguish medical from patient chunks.

9.3 ABBREV_MAP

The module defines 17 regex-based abbreviation expansions applied during text cleaning. These ensure that abbreviations are searchable by their full medical terms, improving retrieval accuracy when physicians query using full clinical terminology:

Abbreviations expanded: **CKD, DM, HTN, MI, HF, COPD, UTI, BP, HR, SpO2, AKI, AF, DVT, PE, ICU, IV**

10. Module: agent_utils.py — Shared Utilities

File: agentt/agent_utils.py **Role:** Support Layer — shared data-access functions for all agent modules.

This module provides the complete set of file-reading utilities used by report_agent.py and indirectly by the chatbot pipeline. It implements the application's file-based data access layer, reading structured CSV files and directory listings.

Function	File/Path Read	Returns
get_active_doctor_folder()	apppy/.active_doctor	Path to active doctor folder or None
require_doctor_folder()	(calls above)	Path or raises PermissionError
list_patients(doctor_folder)	doctor_folder/	Sorted list of patient folder Paths
get_patient_folder(doctor_folder, name)	doctor_folder/<name>/	Validated patient folder Path
load_patient_csv(patient_folder)	<ID>_<First>_<Last>.csv	Dict of registration fields
load_all_diagnostics(patient_folder)	N-vitaldiagnostic.csv (all)	Sorted list of diagnostic dicts
load_home_treatments(patient_folder)	home_treatment.csv	List of treatment plan dicts
load_hospitalized_treatment(patient_folder)	hospitalizedtreatment.csv	Single hospitalisation dict
load_all_check_tables(patient_folder)	Check Table N.csv (all)	Sorted list of check table dicts
list_xrays(patient_folder)	xray/ subdirectory	Sorted list of image Paths
list_uploaded_pdfs(patient_folder)	*_pdf/ subdirectories	Dict: subdir_name -> list[Path]
load_doctor_profile(doctor_folder)	doctor_profile.csv	Dict of doctor fields
parse_json_field(raw)	(in-memory)	Parsed list/dict or [] on failure
parse_patient_name(folder_name)	(in-memory)	Human-readable 'First Last' string

File Naming Conventions

The module uses compiled regex patterns to identify and sort files by their naming convention:

- **PATIENT_FOLDER_RE** = `r'^\d+[_-]'`: Matches patient folders starting with a numeric ID followed by underscore or hyphen (e.g., 1001_Aya_Ahmed).
- **DIAGNOSTIC_RE** = `r'^(ld+)-vitaldiagnostic\l.csv$'`: Captures the appointment index from diagnostic CSVs for chronological sorting.
- **CHECK_TABLE_RE** = `r'^Check Table (ld+)\l.csv$'`: Captures check table numbers for sorted loading.

11. End-to-End Execution Flow

11.1 Application Startup

1. Streamlit launches the main application (app.py or equivalent).
2. The agentt/layer2.py module is imported, triggering MedicalKnowledgeBase instantiation.
3. MedicalKnowledgeBase.build() checks for agentt/med_kb_cache.pkl. If valid: loads in ~1 second. If not: processes all files in agentt/references/ (may take 30-120 seconds depending on corpus size), saves cache.
4. DocumentProcessor is instantiated, loading the SentenceTransformer model (~100MB) into memory.
5. Streamlit session state is initialised (user_kb=None, chat_history=[], processor=DocumentProcessor instance).

11.2 Chatbot Interaction Flow

1. Physician navigates to the Chatbot page in the Streamlit UI.
2. Physician uploads a patient PDF document via the file uploader widget.
3. DocumentProcessor.process_file() parses the PDF, chunks text, builds FAISS and BM25 indexes. The resulting KB dict is saved to st.session_state.user_kb.
4. Physician types a clinical query in the chat input (e.g., 'Is the current antibiotic regimen effective given the worsening trend?').
5. hybrid_retrieve() is called: encodes query, searches user_kb (vector + BM25) and med_kb (vector only), merges candidates, reranks, and returns top 3 PDF chunks + top 3 MED chunks.
6. build_final_prompt() assembles the structured prompt with role definition, output rules, inference rules, doctor query, PDF chunks, and MED chunks.
7. The OpenAI client sends the prompt to the OpenRouter API. Response is streamed and displayed token-by-token in the Streamlit chat interface.
8. Full response is appended to chat_history. Physician continues the conversation; history provides context for follow-up queries.

11.3 Report Generation Flow

1. Physician navigates to the Report Generator page in the Streamlit UI and selects a patient.
2. generate_patient_report(doctor_folder, patient_folder_name) is called.
3. All patient data is loaded via agent_utils.py functions (CSVs, check tables, xray list, PDF list).
4. Doctor profile is loaded from doctor_profile.csv.
5. A ReportLab SimpleDocTemplate is created targeting /agent_report_.pdf.
6. Story elements are built section by section: cover block, page break, then all 6 content sections.
7. doc.build(story, onFirstPage=_footer, onLaterPages=_footer) renders the complete PDF.
8. Function returns (True, 'Report saved to: ', pdf_path). The Streamlit UI displays a download button.

12. Data Flow Diagram (Textual)

The following describes the data flow for the chatbot query pipeline:

```
PHYSICIAN INPUT (Streamlit UI)
|
|--- [Uploaded PDF] --> layer1.py: DocumentProcessor.process_file()
| |
| [FAISS Index + BM25 Index + Chunks]
| |
| st.session_state.user_kb
|
|--- [Text Query] --> retrieval.py: hybrid_retrieve()
| |
| |--- user_kb [FAISS + BM25] --> deduplicated candidates
| |--- med_kb [FAISS only] --> medical candidates
| |
| [Reranker: Cohere / FlashRank / Simple]
| |
| {'pdf_chunks': [...], 'med_chunks': [...]}
|
prompt_builder.py: build_final_prompt()
|
[Structured Prompt String]
|
OpenRouter API (openai client)
|
[Streamed LLM Response]
|
Streamlit Chat Interface (displayed + saved to history)
```

13. Dependencies & Environment Requirements

All dependencies are declared in `agentt/requirements.txt`. The following table details each package, its minimum version, and its role in the system:

Package	Min Version	Role in System
streamlit	>=1.35.0	Web UI framework — provides chat interface, file uploaders, buttons, session state
openai	>=1.30.0	LLM API client — configured to call OpenRouter's OpenAI-compatible endpoint
python-dotenv	>=1.0.0	Environment variable loading — reads <code>OPENROUTER_API_KEY</code> and <code>COHERE_API_KEY</code>
pypdf	>=4.0.0	PDF text extraction — used in <code>layer1.py</code> and <code>med_kb.py</code> to read PDF files
sentence-transformers	>=3.0.0	Dense text embedding — provides 'all-MiniLM-L6-v2' model for semantic encoding
faiss-cpu	>=1.8.0	Vector similarity search — IndexFlatL2 for exact L2 nearest-neighbour search
rank-bm25	>=0.2.2	Sparse keyword search — BM25Okapi implementation for term-frequency retrieval
flashrank	>=0.2.9	Local cross-encoder reranking — no API key required; used as primary reranker
numpy	>=1.26.0	Numerical arrays — required for FAISS index operations and embedding matrices
cohere	>=5.0.0	Optional cloud reranker — Cohere Rerank API; only used if <code>COHERE_API_KEY</code> is set
reportlab	(implied)	PDF generation — used in <code>report_agent.py</code> for all report layout and rendering
langchain-text-splitters	(implied)	Text chunking — RecursiveCharacterTextSplitter in <code>layer1.py</code>

Note: flashrank and cohere are mutually exclusive runtime paths. The system selects the best available reranker at runtime using the graceful degradation pattern. If neither is available or their initialisation fails, `_rerank_simple()` is used as a zero-dependency fallback.

14. Configuration & API Keys

14.1 Required Environment Variables

Variable	Required?	Used By	Description
OPENROUTER_API_KEY	YES	layer2.py	API key for OpenRouter LLM gateway. Obtain at openrouter.ai
COHERE_API_KEY	NO (optional)	retrieval.py	API key for Cohere Rerank API. If absent, FlashRank is used i

14.2 .env File Setup

Create a file named `.env` in the project root directory:

```
OPENROUTER_API_KEY=sk-or-v1-xxxxxxxxxxxxxxxxxxxxxxxxxxxxx
COHERE_API_KEY=xxxxxxxxxxxxxxxxxxxxxxxxxxxxx # optional
```

14.3 Key File Paths (Hardcoded Relative to Module Location)

Path	Module	Purpose
agentt/refsrences/	med_kb.py	Source PDFs and TXTs for medical knowledge base
agentt/med_kb_cache.pkl	med_kb.py	Serialised cache of MedKB chunks and FAISS index
apppy/.active_doctor	agent_utils.py	Text file containing active doctor folder path
<doctor_folder>/doctor_profile.csv	agent_utils.py	Doctor registration information
<patient_folder>/<ID>_<Name>.csv	agent_utils.py	Patient registration record
<patient_folder>/N-vitaldiagnostic.csv	agent_utils.py	Diagnostic session records
<patient_folder>/home_treatment.csv	agent_utils.py	Home treatment plan records
<patient_folder>/hospitalizedtreatment.csv	agent_utils.py	Hospitalisation record
<patient_folder>/Check Table N.csv	agent_utils.py	Nurse monitoring check tables
<patient_folder>/xray/	agent_utils.py	X-ray image storage directory
<patient_folder>/agent_report_*.pdf	report_agent.py	Generated PDF reports

15. Medical Knowledge Base (references/ folder)

The agentt/refs/ directory is the source for the Medical Knowledge Base. In the current version (v9), it contains two peer-reviewed medical papers:

File	Title	Domain
diagnostics-15-02244.pdf	Diagnostic Innovations to Combat Antibiotic Resistance in Acute Care (Alstam et al, 2015, Diagnostics)	
ijerph-17-06278.pdf	The Characteristic of Virulence, Biofilm and Antibiotic Resistance of <i>Klebsiella pneumoniae</i> Isolates from Hospital Outpatients	

These papers were selected to support the application's focus on antibiotic resistance diagnosis, relevant to the management of pediatric pneumonia caused by resistant pathogens such as *Klebsiella pneumoniae*. After processing, the two papers generate multiple text chunks stored in med_kb_cache.pkl, each tagged with 'MED' and the source paper's filename stem.

Extending the Knowledge Base

To add new medical literature: place additional .pdf or .txt files in agentt/refs/ and delete agentt/med_kb_cache.pkl. On next application start, MedicalKnowledgeBase.build() will detect the cache is stale (hash mismatch) and reprocess all files automatically.

16. Security & Data Handling

- **API Key Security:** API keys are loaded exclusively from environment variables via python-dotenv. They are never hardcoded in source files and never transmitted to clients.
- **Patient Data Isolation:** All patient data access is gated by the active doctor session file. The require_doctor_folder() function raises PermissionError if no session is active, preventing unauthorised data access.
- **Local Processing:** Document embedding, BM25 indexing, and FlashRank reranking are performed entirely locally. Patient document content is NOT sent to external APIs during indexing.
- **LLM Data Transmission:** When the chatbot is used, retrieved text chunks from patient documents ARE included in the prompt sent to the OpenRouter API. Physicians should be aware that this involves transmitting patient-relevant content to a third-party API endpoint.
- **Pickle Cache Security:** The med_kb_cache.pkl file is a Python pickle. It should not be loaded from untrusted sources. In this implementation, it is only written and read by the application itself.
- **No Database:** All data is stored in flat CSV files. There is no SQL injection risk. File path traversal risks are mitigated by regex validation of folder names in agent_utils.py.

17. Limitations & Future Work

17.1 Current Limitations

- **LangChain Dependency in layer1.py:** layer1.py imports langchain_text_splitters, which is not listed in requirements.txt. This creates a hidden dependency that may cause ImportError on fresh installs.
- **Single-Session KB:** The user knowledge base (uploaded PDF) is stored in Streamlit session state and lost on browser refresh. There is no persistent per-patient PDF knowledge base.
- **No Conversation Context in Retrieval:** The retrieval pipeline only uses the current query for search. It does not incorporate conversation history, which may reduce retrieval quality for follow-up questions.
- **MedKB Size Limitation:** The current MedKB contains only 2 papers. Clinical coverage is narrow. The system performs best when the references/ directory contains comprehensive, domain-specific literature.
- **LLM Response Consistency:** LLM outputs are non-deterministic. The same query may produce different responses across sessions. The structured prompt reduces but does not eliminate this variability.
- **X-ray Integration:** X-ray images are listed in reports but not embedded. The chatbot does not perform image analysis; radiological assessment requires separate tools.
- **No Authentication in Agent Layer:** The agent layer relies on the application-level session file for authentication. It does not implement independent access control.

17.2 Recommended Future Enhancements

- Add langchain-text-splitters to requirements.txt to resolve the hidden dependency.
- Implement persistent per-patient PDF knowledge bases stored in the patient folder.
- Add conversation history to retrieval context for improved follow-up query handling.
- Expand the references/ directory with paediatric pneumonia guidelines (WHO, BTS, IDSA).
- Integrate X-ray image analysis using a DICOM-compatible vision model.
- Implement multi-language support for clinical notes and reports (Arabic for Algerian context).
- Add a formal evaluation framework measuring retrieval precision and LLM response clinical accuracy.
- Replace pickle-based cache with a proper vector database (e.g., ChromaDB, Qdrant) for scalability.

Module Summary Reference

Module	Lines (approx.)	Layer	Key Dependency	Output
layer1.py	~60	Ingestion	faiss, rank-bm25, flashrank	KB dict {chunks, index, bm25}
retrieval.py	~120	Retrieval	faiss, cohere, flashrank	{pdf_chunks, med_chunks}
prompt_builder.py	~70	Reasoning	(none)	Structured prompt string
layer2.py	~100	Generation	openai, streamlit	Streamed LLM response
report_agent.py	~280	Reporting	reportlab	Timestamped PDF file
med_kb.py	~120	Support	faiss, pypdf, sentence-transformers	Indexed MedKB + FAISS index
agent_utils.py	~180	Support	csv, json, pathlib	Typed data dicts/lists

